

Vektor- datenbanken



Wortvektoren, Ähnlichkeitssuche und Hybrid-Retrieval

Wozu brauchen wir Vektoren?

Motivation

Klassische Suche findet exakte Übereinstimmungen.

Für eine semantische Suche auf unserer Wissensbasis müssen wir hingegen die **Bedeutung** von Wörtern vergleichen.

→ Also beurteilen können, dass „Kosten der Produktion“ und „Herstellungsaufwand“ dasselbe bedeuten.

Dafür kodieren wir Texte als **Vektoren** in einem hochdimensionalen Bedeutungsraum; und berechnen **Nähe** statt Gleichheit.

Google-Suche

Semantische Rankings seit BERT (2019) [Q1]

Empfehlungs-Algorithmen

z.B. TikTok For You Page [Q2]

Empfehlungen über Nutzer- & Content-Vektoren

Moderne LLMs

Grundbaustein [Q3]

Unser Projekt

Semantische Dokumentensuche für RAG

Wie entstehen Wortvektoren?

Überblick



Trainingsdaten

Milliarden Sätze aus Textkorpora
(Hauptsächlich Internet) [Q4]



Training

Modell lernt aus den Daten, Wörter
aus ihrem Kontext vorherzusagen.



Embedding

Fixe Vektor-Repräsentation pro
Textabschnitt, 256-3072 Dim. [Q3][Q4]

Schlüsselidee: Einzelne Dimensionen sind nicht menschenlesbar – Bedeutung entsteht aus der *geometrischen Lage* der Vektoren zueinander. Ähnliche Texte liegen nah beieinander.

Wie entstehen Wortvektoren?

Training

2

Training

Modell lernt aus den Daten, Wörter aus ihrem Kontext vorherzusagen.

Selbstüberwachtes Lernen (Masking)

Neue Vektor-Modelle lernen durch das Weglassen & Raten von Wörtern. [Q16]

Der ██████ bellt heute laut im Garten.

→ Dadurch lernen sie implizit die Bedeutung von Wörtern im Kontext

Attention

Durch Attention lernen Modelle, wie Wörter sich gegenseitig beeinflussen. [Q3][Q16]

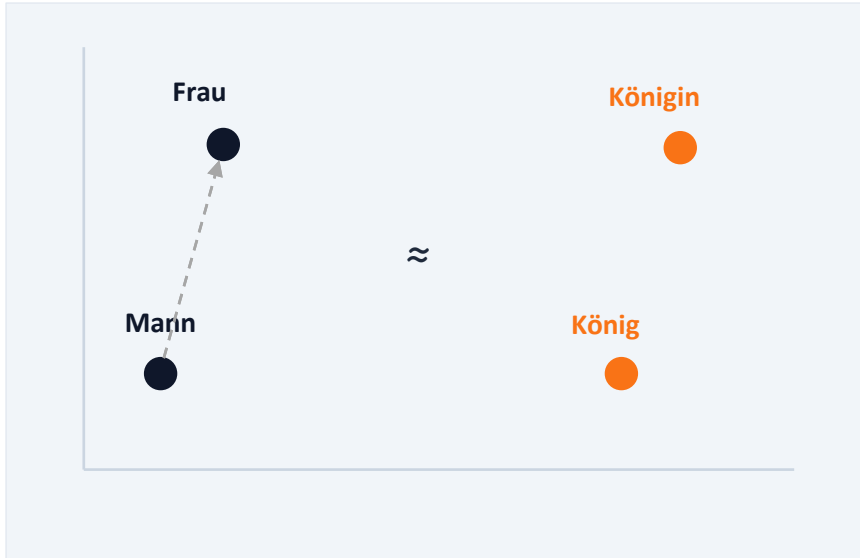
Prince Harry Potter

→ Dadurch werden Wörter abhängig von ihrer Umgebung unterschiedlich gewichtet



Bedeutung als Geometrie

Verständnisbeispiel



Semantische Richtungen

Beziehungen zwischen Begriffen entsprechen

Vektordifferenzen. Der Vektor von „Mann $\rightarrow$ König“ ist fast identisch zum Vektor „Frau $\rightarrow$ Königin“. [Q5]

Analog:

Deutschland $\rightarrow$ Japan $\approx$ Bratwurst $\rightarrow$ Sushi

Vektordatenbank vs. relationale DB

Abgrenzung

	Relationale DB	Vektordatenbank
Datentyp	Tupel / Tabelle	Vektor + Payload
Suche	Exakte Übereinstimmung	Ähnlichkeit (Distanz)
Abfrage	SQL WHERE, JOIN	k-Nearest-Neighbor
Index	B-Tree, Hash	HNSW, IVF, PQ
Ergebnis	Boolesche Treffermenge	Nach Relevanz sortierte Liste

Beispieleintrag in einer Vektor-DB

```
{
  "id": "chunk_0421",
  "vector": [0.21, -0.05, ...],
  "payload": {
    "text": "Die Produktionskosten...",
    "quelle": "reportQ3.pdf"
  }
}
```

[Q6]

Vektor-DBs beantworten die Frage „**Was ist ähnlich?**“, nicht „Was ist gleich?“.

Retrieval

Ingestion & Retrieval – Ein Überblick mit Vektor-DB

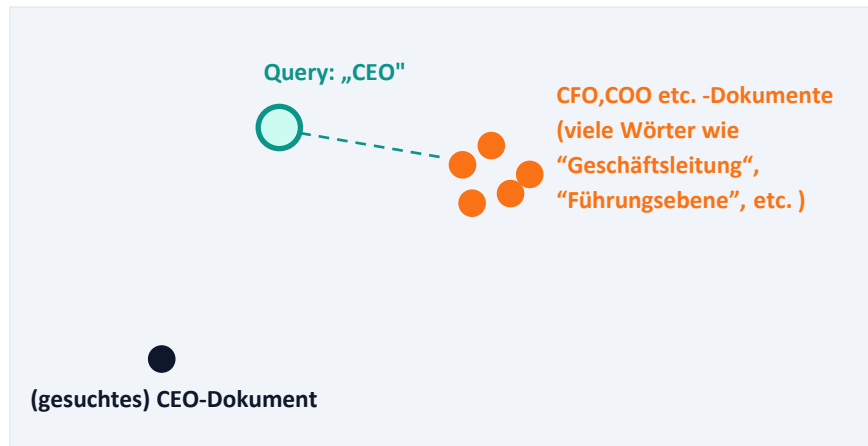
[Web-Illustration](#)

Wo Vektorsuche allein versagt

Blind-Spots

Szenario:

Query: „CEO“



CFO-Dokumente mit hoher Dichte an „CEO-ähnlichen“ Wörtern liegen im Vektorraum näher an der Query als die eigentlichen CEO-Dokumente.

→ Können in der Praxis höher ranken. [Q11]

Andere Typische Schwachstellen

- Seltene Fachbegriffe, IDs, Codes [Q15]
z. B. Produkt-Code „X-42/B“ hat keine gelernte Semantik.
- Negationen [Q14]
„nicht toxisch“ liegt nahe an „toxisch“.

Lösung: Hybrid-Retrieval [Q8]

Vektorsuche mit Keyword-Verfahren (BM25 / FTS)
kombinieren und Ergebnisse gewichtet fusionieren. (RRF)

Bonus: explizite Ausschlüsse.

`vector_query + bm25_query - excluded_keywords`

Architekturvorschlag

Ingestion & Retrieval-Tool



Retrieval-Tool für den Chatbot

```
search({  
  vector_query:      "Kündigungsfrist"  
  bm25_query:        "$ 622 BGB"  
  vector_weight:     0.2  
  excluded_keywords: ["Entwurf"]  
})
```

Warum diese Wahl?

- Qwen3-Embedding:
Top-Werte im MTEB-Benchmark für DE/EN [Q9]
- Lokaler Betrieb:
→ keine API-Kosten, DSGVO-konform
- Qdrant ist hybrid aus einer Hand - kein zusätzlicher Suchindex [Q10]

Anhang

CLIENT

RETRIEVAL-ENGINE

WISSENSBASIS

LLM / Chatbot

z. B. GLM, Qwen, Gemma

erzeugt über Retrieval-Tool

Vektor-Query

Keyword-Query

sparse query

Embedding-Modell

Qwen3-Embedding-0.6B

dense vector (1024-dim)



Hybrid Vector Database - HNSW-Index + BM25

QUERYING

INDEX / CHUNKING

BM25
Keyword-Ranking

Embedding
Vektor-Ranking

Finales Ranking
Weighted RRF

Weighted RRF

Top-k Chunks → LLM

NATIVE DOKUMENTE

Chunking

Z. B. CONFLUENCE

ANDERE

indizierte Chunks

Anhang

Quellen

Q1 - Google BERT Blogpost

<https://blog.google/products/search/search-language-understanding-bert/>

Q2 - TikTok / Monolith (ByteDance, Liu et al. 2022)

<https://arxiv.org/abs/2209.07663>

Q3 - Embeddings als Grundbaustein von LLMs: Attention Is All You Need

<https://arxiv.org/abs/1706.03762>

Q4 - Aktuelle Embedding-Modelle mit 3072 Dimensionen + Matryoshka-Shortening (256 Dims)

<https://openai.com/index/new-embedding-models-and-api-updates/>

Q5 - Mikolov et al. 2013 (word2vec)

<https://arxiv.org/abs/1301.3781>

Q6 - Qdrant Point-Struktur

<https://qdrant.tech/documentation/manage-data/points/>

Q7 - HNSW (Malkov & Yashunin)

<https://arxiv.org/abs/1603.09320>

Q8 - Hybridsuche mit BM25 (Thakur et al. 2021, BEIR-Benchmark, NeurIPS)

<https://arxiv.org/abs/2104.08663>

Q9 - MTEB & Leaderboard

Paper: <https://arxiv.org/abs/2210.07316>

Live-Leaderboard: <https://huggingface.co/spaces/mteb/leaderboard>

Q10 - Qdrant Hybrid-Suche mit BM25 nativ

<https://qdrant.tech/documentation/tutorials-and-examples/cloud-inference-hybrid-search/>

Q11 – Eigenreproduktion CEO/CFO: Siehe Anhang/Github

Q12 – ChromaDB Hybrid Sparse Vector Support

<https://www.trychroma.com/project/sparse-vector-search>

Q13 – VectorChord Github & Readme

<https://github.com/tensorchord/VectorChord-bm25>

Q14 – Vektorsuche Negation Blind-Spot

<https://arxiv.org/abs/2603.17580>

Q15 – Vektorsuche Out of Vocabulary / Out of Domain
Blind-Spot

<https://dev.to/gabrielanhaia/rag-without-embeddings-when-bm25-beats-your-020-per-1k-vector-index-2140>

Q16 – Masking&Attention als Training-Strategie für
Embedding-Modelle

<https://arxiv.org/abs/1810.04805>

Anhang

Empirischer Test der Vektorsuche-Schwäche aus Folie 7

Setup

Query: "CEO" · **Korpus:** 8 Chunks (1× CEO-Profil, 7× andere Führungsrollen ohne den Begriff „CEO“) · **Bewertung:** Failure, wenn das CEO-Dokument nicht in den Top-3 nach Kosinusähnlichkeit landet.

openai · text-embedding-3-large

FAILURE

Wanted-Rank: 4 / 8

#1	0.397	Operations-Chunk
#2	0.367	Chief Revenue Officer
#3	0.316	Chief Product Officer
#4	0.311	CEO-Dokument ← Ziel

openai · text-embedding-3-small

FAILURE

Wanted-Rank: 8 / 8

#1	0.385	Operations-Chunk
#2	0.329	Chief Product Officer
#3	0.306	Finance-Chunk
...		
#8	0.284	CEO-Dokument ← Ziel

google · gemini-embedding-001

PASS

Wanted-Rank: 1 / 8

#1	0.652	CEO-Dokument ← Ziel
#2	0.642	Operations-Chunk
#3	0.633	Gründer-Chunk
#4	0.623	Chief Revenue Officer

Befund: Failure ist modellabhängig - bei 2 von 3 getesteten Modellen reproduzierbar. Gemini schneidet besser ab, weil es Query und Dokument asymmetrisch enkodiert (RETRIEVAL_QUERY vs. RETRIEVAL_DOCUMENT). Das Risiko genügt, um in der Architektur durch Hybridsuche adressiert zu werden.

Anhang

Vergleich Vektordatenbanken – engere Auswahl auf Basis des Open-Source-Kriteriums

Kriterium	Qdrant	ChromaDB	pg_vector
Architektur	Native Vektordatenbank	Native Vektordatenbank	Postgres-Erweiterung
Lizenz	Apache 2.0	Apache 2.0	PostgreSQL Lic.
Implementiert in	Rust	Python / Rust	C
Hybridsuche (BM25)	Nativ unterstützt [Q10]	Neuerdings nativ unterstützt [Q11]	Über Plugin (VectorChord-BM25) [Q12]
Filterung über Metadaten	Ja	Ja	Ja (über SQL)
Self-Hosting	Ja	Ja	Ja
Reife / Verbreitung	Etabliert	Etabliert	Semi-Etabliert – VectorChord ist noch neuartig

Fazit

Qdrant unterstützt Hybridsuche aus Vektor- und BM25-Komponente nativ und produktionsreif. ChromaDB bietet seit neuestem ebenfalls native Hybridsuche, ist allerdings nicht so performant wie Qdrant; pg_vector setzt auf neuartige Plugins wie VectorChord-BM25, deren Reifegrad für unseren Anwendungsfall noch unklar ist.
-> Alle Optionen könnten eine valide Option für unser Projekt darstellen.

Anhang

Vergleich Embedding-Modelle: Auswahl auf Basis von MTEB / RTEB-Multilingual (Retrieval)

Modell	Anbieter	Parameter	Dimensionen	Zero-Shot	nDCG@10 ↑	Rang
Qwen3-Embedding-8B	Alibaba	8.0 B	4096	99 %	70.88	2
Harrier-OSS-v1-0.6B	Microsoft	0.6 B	1024	78 %	70.75	3
Qwen3-Embedding-4B	Alibaba	4.0 B	2560	99 %	69.60	5
Qwen3-Embedding-0.6B	Alibaba	0.6 B	1024	99 %	64.84	14

★ Empfehlung: Qwen3-Embedding-0.6B

Bestes Verhältnis aus Retrieval-Qualität, Modellgröße und Inferenzkosten. Liefert **91 %** **der nDCG@10** des 13× größeren 8B-Geschwistermodells bei deutlich geringerem Speicher- und Compute-Bedarf - relevant für Ingestion großer Dokumentmengen.

Methodische Anmerkungen

Zero-Shot = Modell wurde nicht auf den Trainingsdaten des Benchmarks trainiert. Ein hoher Wert ist Voraussetzung für aussagekräftige Vergleiche.

Unbekanntere Spitzenmodelle (zb. Octen-Embedding) wurden ausgeschlossen: hohe Werte, aber kaum Adoption – evtl. Risiko von Benchmark-Overfitting.